Penetration Testing

Rocksolid (DeFi)

September 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

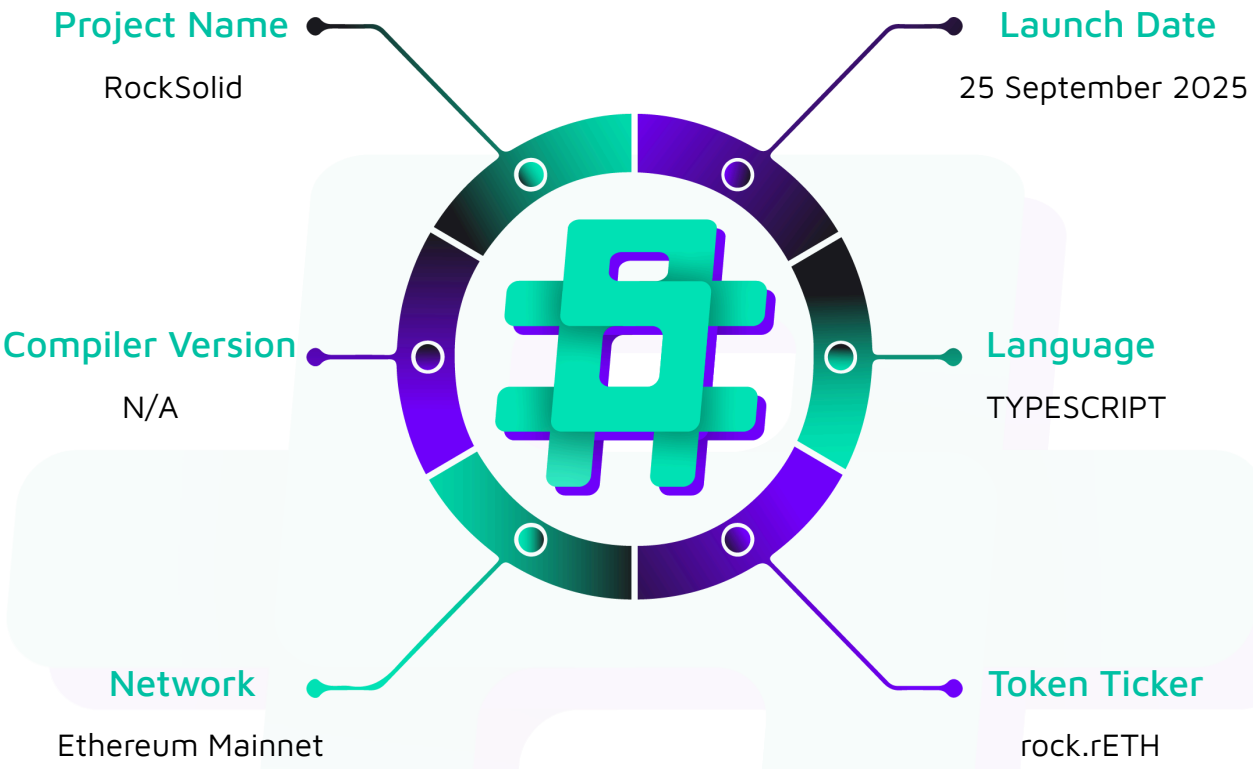
The RockSolid team partnered with Hashlock to conduct a security Penetration Test of their RockSolid Project code base, and an analysis and verification of the MPC wallet setup and policy controls. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

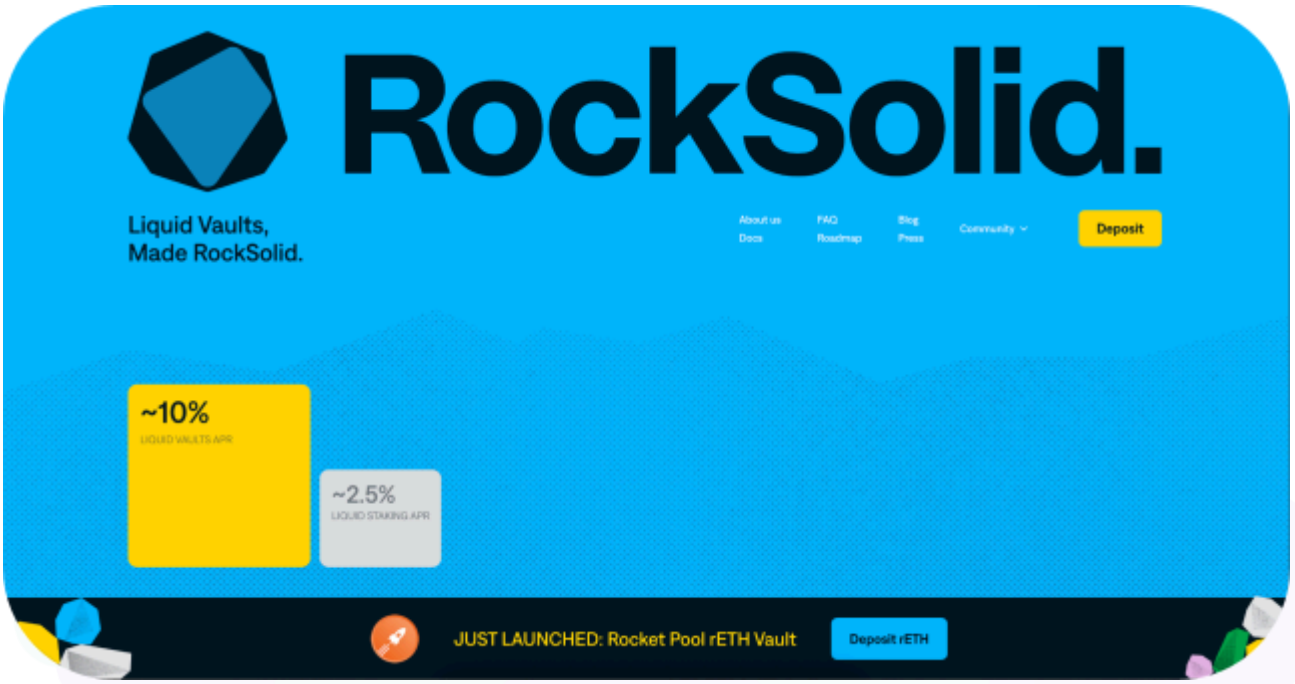
RockSolid is a decentralized finance (DeFi) platform focused on providing institutional-grade liquid vaults for crypto asset depositors and protocol partners. It enables users to access curated DeFi strategies and optimized yield opportunities through a simplified, non-custodial interface, while offering protocols white-label vault infrastructure to bootstrap liquidity and manage risk. The platform integrates across multiple DeFi protocols and asset types, allowing users to earn yield without manually staking, bridging, or monitoring positions. From a security and governance standpoint, RockSolid's architecture relies on smart contracts interacting with third-party protocols, introducing typical DeFi risks such as contract vulnerabilities, oracle dependencies, liquidity risk, and de-peg exposure. The platform emphasizes user key ownership and transparent strategy management, with an operational model designed to balance accessibility and security.

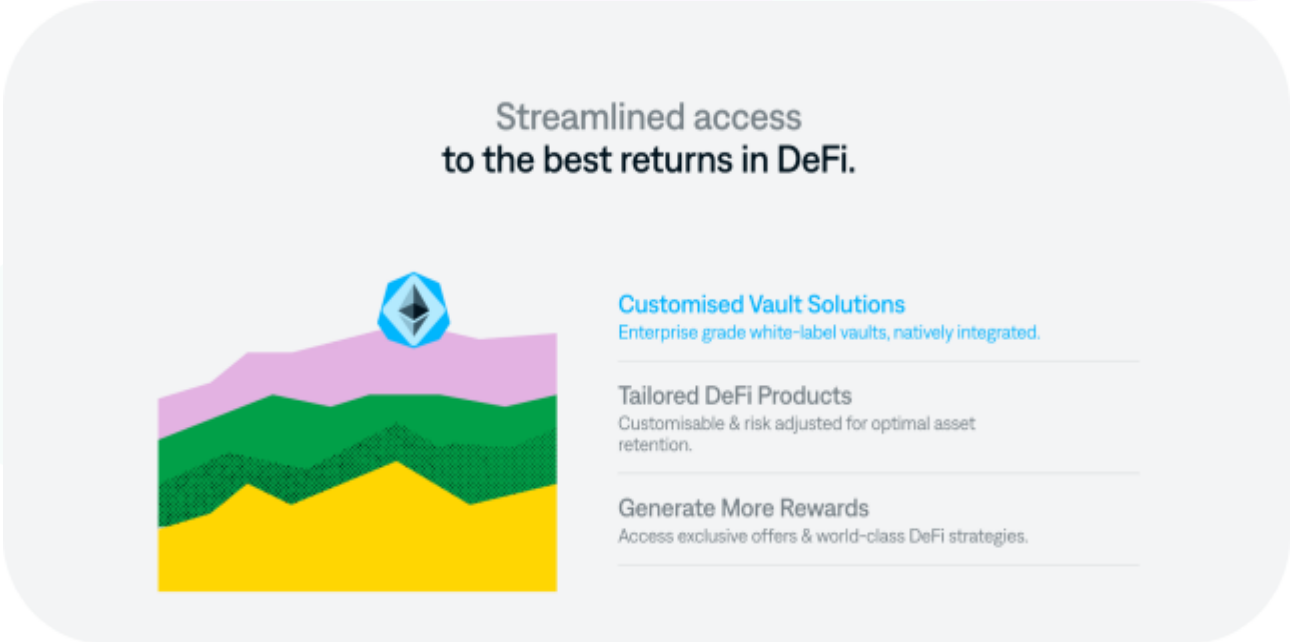
Project Name: RockSolid
Project Type: DeFi
Website: https://rocksolid.network/
Logo:

Visualised Context:



Project Visuals:





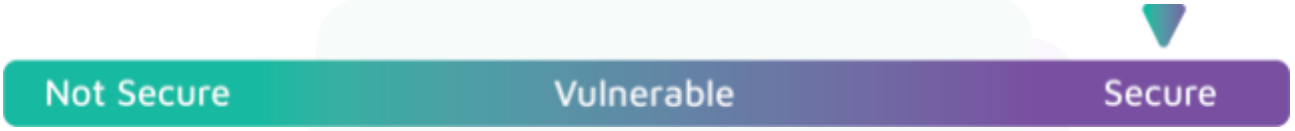
Penetration Test of Web App Scope

At Hashlock, we executed a comprehensive penetration test on the RockSolid project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**RockSolid Project Code Base**
Platform	**Web apps & APIs**
Penetration Test Date	**September, 2025**
Repository/Website URL/IP Tested	The following URLs point to the same application - https://vip.rocksolid.network/ - https://app.rocksolid.network/
Repository	https://github.com/rocksolid-network/rocksolid-vaults

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



All issues uncovered during automated and manual analysis were meticulously reviewed, and applicable vulnerabilities are presented in the Penetration Test Findings section.

We have identified some vulnerabilities that need to be addressed prior to launch.

Hashlock found:

1 Medium-severity vulnerability

4 Low-severity vulnerabilities

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test of Web App Findings

Medium

[M-01] next.config.js - Server-Side Request Forgery (SSRF) via Remote Image Provider Takeover

Description

The Next.js image optimization configuration whitelists several Cloudflare R2 bucket hostnames that may be unclaimed or have been deleted. An attacker can register one of these unclaimed hostnames and use the Next.js image endpoint to force the application server to make arbitrary requests on their behalf.

Vulnerability Details

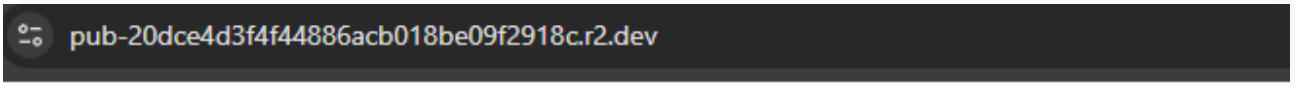
The images.remotePatterns array in `next.config.js` explicitly trusts domains like `pub-20dce4d3f4f44886acb018be09f2918c.r2.dev`. The Next.js Image Optimization service (`/_next/image`) will fetch images from any URL that matches these patterns. If an R2 bucket with this name does not exist, an attacker can claim it. By hosting a malicious file (e.g., one that redirects to an internal IP address) on the newly claimed bucket, the attacker can use the `/_next/image` endpoint as a proxy to make the server perform an SSRF attack. The `dangerouslyAllowSVG: true` setting further elevates the risk, as an attacker could serve malicious SVGs to achieve Stored XSS.

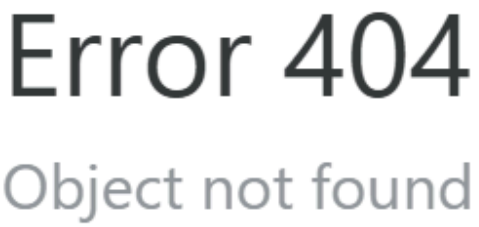
```
const nextConfig: NextConfig = {
  /* config options here */
  experimental: {
    optimizePackageImports: ["recharts", "lucide-react", "thirdweb", "viem"],
    scrollRestoration: true,
  },
  images: {
```

```
  dangerouslyAllowSVG: true,
  remotePatterns: [
    {
      protocol: "https",
      hostname: "www.citypng.com",
      pathname: "/**",
    },
    {
      protocol: "https",
      hostname: "pub-20dce4d3f4f44886acb018be09f2918c.r2.dev",
      pathname: "/**",
    },
    {
      protocol: "https",
      hostname: "s2.coinmarketcap.com",
      pathname: "/**",
    },
    {
      protocol: "https",
      hostname: "basescan.org",
      pathname: "/**",
    },
    {
      protocol: "https",
      hostname: "pub-5c5ffd799df84066bfdcc9b1674db989.r2.dev",
      pathname: "/**",
```

```
      },
      {
        protocol: "https",
        hostname: "pub-d1e588c29e154821be7d4edfda815bd5.r2.dev",
        pathname: "/**",
      },
    ],
  },
};
```

The URLs were checked for existence:





Potentially, an attacker can claim a bucket using that name to be able to load content to the RockSolid web application.

Impact

This vulnerability allows for a blind SSRF attack, which can be used to scan the application's internal network, probe for open ports, or interact with internal-only

services. In a cloud environment, this could lead to the exposure of sensitive cloud metadata credentials. The secondary impact is Stored XSS if a user can be tricked into viewing a link to a malicious SVG processed by the image optimizer.

Recommendation

Immediately audit all hostnames listed in the images.remotePatterns configuration. Verify that each domain is active, claimed, and controlled by your organization. Remove any domains that are unused, unclaimed, or correspond to deleted resources.

Status

Resolved

Low

[L-01] cloudflare-env.d.ts - Risk of sensitive information disclosure

Description

The `cloudflare-env.d.ts` file declares that `NEXT_PUBLIC_CACHE_ADMIN_SECRET` exists as an environment variable. `NEXT_PUBLIC_` prefix explicitly signals to the Next.js framework - and to developers - that the variable is safe to be embedded directly into client-side JavaScript. At the same time, there is also a non-public variant of it.

Due to the application's current structure, no client-side component actually references this public-prefixed variable. As a result, the Next.js build process is performing dead-code elimination and correctly stripping the variable from the final client-side bundles. Local build analysis confirms the secret is not present in the `.next/static/` directory.

However, during application development, this may happen where a developer may expose the secret, seeing that it should be public based on the `NEXT_PUBLIC` prefix.

Recommendation

Remove the `NEXT_PUBLIC_CACHE_ADMIN_SECRET` environment variable entirely. There is no legitimate use case for a public-facing version of a cache administration secret.

Status

Resolved

[L-02] API - Verbose error messages

Description

Several API endpoints return detailed internal error messages to the client. This can leak information about the application's internal workings, library versions, and error states, which can aid an attacker in building a more targeted exploit.

In catch blocks, error objects from third-party APIs or internal exceptions are directly stringified, or their messages are passed into the JSON response. For example, in `src/lib/helpers/swap.ts`, the full error object from the API is included in the details field of the error response. A similar pattern exists in `src/app/api/admin/cache/invalidate/route.ts`. While not directly exploitable, this information disclosure provides attackers with valuable reconnaissance data.

Recommendation

Implement a centralized error handling utility that catches all errors and returns a generic, sanitized error message to the client while logging the detailed error on the server for debugging.

Status

Resolved

[L-03] route.ts - User-Agent blocklist can be bypassed

Description

In `src/app/api/public/swap/quote/route.ts`, the `BLOCKED_USER_AGENTS` array contains a list of regular expressions that are checked against the User-Agent header of incoming requests. An attacker can circumvent this check by simply changing their User-Agent header to a standard browser string (e.g., "Mozilla/5.0...").

Recommendation

Remove the User-Agent blocklist and rely on more reliable security controls like CloudFlare's rate limiting.

Status

Resolved

[L-04] package.json - Vulnerable dependencies

Description

A static scan using `Snyk` tool revealed the following vulnerable dependencies. While this does not mean they are exploitable, all dependencies should be updated and patched to minimize the attack surface.

```
Issues to fix by upgrading:

  Upgrade next@15.3.3 to next@15.4.2 to fix
    ✗ Missing Source Correlation of Multiple Independent Data (new) [Low Severity][https://security.snyk.io/vuln/SNYK-JS-NEXT-12265451] in next@15.3.3
   introduced by next@15.3.3
      ✗ Use of Cache Containing Sensitive Information (new) [Medium Severity][https://security.snyk.io/vuln/SNYK-JS-NEXT-12301496] in next@15.3.3
   introduced by next@15.3.3
          ✗ Server-side Request Forgery (SSRF) (new) [High Severity][https://security.snyk.io/vuln/SNYK-JS-NEXT-12299318] in next@15.3.3
   introduced by next@15.3.3
```

Recommendation

Update the dependencies to the latest, secure versions.

Status

Resolved

Contract Verification Statement

Our verification confirms that the deployed contract matches the version that was audited. The implementation contract at **0xe50554ec802375c9c3f9c087a8a7bb8c26d3dedf** corresponds to the audited commit **0ca582d2ea92504b01032c317e98e6fd7ac6b871** and is linked to the proxy contract at **0x936facdf10c8c36294e7b9d28345255539d81bc7.**

Link to the Github contract and audited commit from the Nethermind Report:

https://github.com/hopperlabsxyz/lagoon-v0/blob/0ca582d2ea92504b01032c317e98e6fd7ac6b871/src/v0.5.0/Vault.sol

Link to the proxy contract on Etherscan:

https://etherscan.io/address/0x936facdf10c8c36294e7b9d28345255539d81bc7#code

Link to the Implementation contract on Etherscan

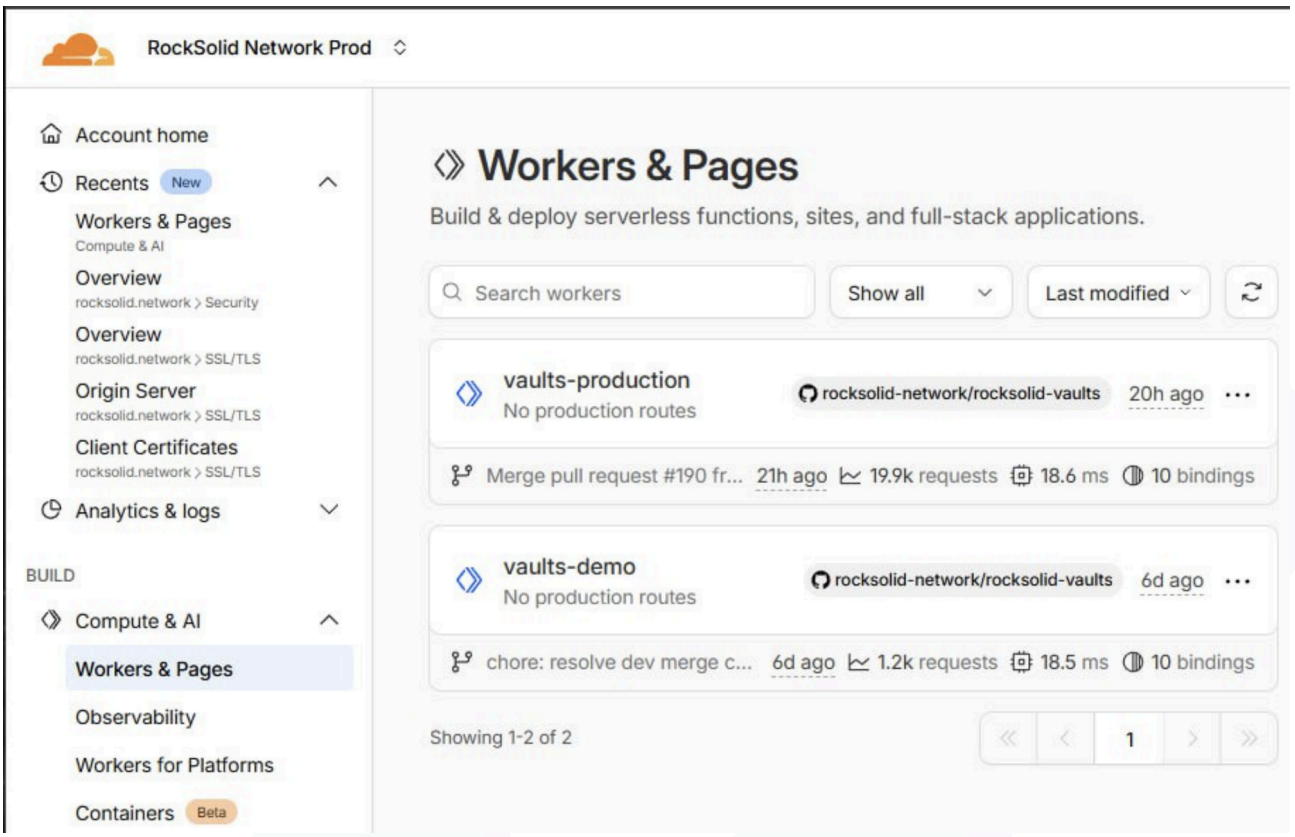
https://etherscan.io/address/0xe50554ec802375c9c3f9c087a8a7bb8c26d3dedf#code

Cloudflare Analysis and Verification

All URLs—https://vip.rocksolid.network/, https://app.rocksolid.network/, and https://rocksolid.network/—resolve to the same IP address and load the same deployed application instance. This ensures consistency in user access and mitigates the risk of discrepancies across different entry points. This can be evidenced in the screenshot below:

The web application is deployed through Cloudflare, which is connected directly to the audited GitHub repository. On each merge to the main branch, a build is automatically triggered in Cloudflare, and the resulting deployment ensures that the live application reflects the verified code. This deployment workflow prevents unauthorized

modifications to the application, including critical parameters such as contract deposit addresses. Any attempt to alter the application would require a compromise of either the Cloudflare account or the GitHub repository, mitigating the risk of a depositor being redirected to a malicious contract. The following confirm this conclusion:



The Cloudflare configuration confirms that the linked GitHub repository is https://github.com/rocksolid-network/rocksolid-vaults, which is correctly integrated with the development workers. This linkage ensures that Cloudflare deployments are directly built from the specified repository, maintaining consistency between the source code and the deployed environment.

Cloudflare Configuration and Security Controls

As part of the audit, a deeper analysis was performed on the client's Cloudflare configuration to assess the security and deployment hygiene of the application's edge infrastructure. The following observations were made:

Access Management

- **Multi-Factor Authentication (MFA):** Enforced for all users.
- **Principle of Least Privilege:** Administrative and regular user roles are properly segregated.

Deployment Integrity

- **Workers & Pages:** The build is correctly linked to the intended repository, ensuring source integrity and traceability.

Application Security

- **Web Application Firewall (WAF):** Enabled and properly configured, adding an additional layer of protection against common web-based attacks.
- **Rate Limiting:** Implemented across relevant endpoints, helping to mitigate risks of denial-of-service (DoS) or brute-force attempts.

Network Security

- **SSL/TLS:** Properly configured, ensuring secure data transmission between clients and servers.

Analysis & Methodology of the MPC

MPC Policy Compliance Review

The Admin Quorum policy is configured to require approvals from 2 of 3 authorized groups: **RockSolid Admins (Distributor)**, **Tulipa (Asset Manager)**, and **the Security Council.**

In this structure, the Security Council serves as a contingency mechanism designed to preserve operational continuity in the event of a critical failure—such as the incapacitation, disablement, or loss of access of either the asset manager or distributor. Should one of these primary entities become unable to participate in the MPC signing process, the Security Council can substitute its approval to maintain the required 2-of-3 quorum and ensure that funds remain accessible and governance processes continue without disruption.

Based on the reviewed MPC policies, the Net Asset Value (NAV) update process follows a multi-step flow involving both the asset manager (Tulipa) and the distributor (RockSolid).

- **Proposal and Expiration**:
  - **Policy 70** defines that Tulipa, as the asset manager, is solely authorized to propose a new NAV value (`updateNewTotalAssets`).
  - **Policy 69** specifies that Tulipa is also responsible for expiring a previous NAV (`expireTotalAssets`), which similarly requires only Tulipa's approval.

- **Settlement and Acceptance**:
  - **Policies 71** and **72** govern the settlement of redemptions and deposits following a NAV update (`settleRedeem` and `settleDeposit`).
  - These actions **require dual-party approval**, with one signer from RockSolid (the distributor) and one from Tulipa (the asset manager).

In summary, while the initial NAV proposal and expiration actions are single-signer operations managed by the asset manager, the critical settlement actions enforcing the updated NAV are properly protected by a two-party approval structure.

Whitelisting and Deployment of New Strategies – Dual Approval Verification

Implementing a new or complex strategy or upgrading the vault contracts involves two distinct but related governance processes:

- **Contract Upgrades:**
  - **Policy 79** (`submitImplementation`) and Policy 80 (`upgradeAndCall`) define how vault contracts are upgraded when Lagoon publishes a new version (for example, to add features or apply a bug fix).
    Opting into these upgrades requires 2-party approval — one signer from **RockSolid Admins (the distributor)** and one from **Tulipa (the asset manager)**

- **Whitelisting and Policy Changes:**
  - Any modification to the existing policies, including adding or changing a strategy, requires Admin Quorum approval. This ensures both Tulipa and RockSolid approve before a change takes effect.
  - The execution of a whitelisted policy then follows the specific signing requirements defined within that policy (some require dual signatures, while others do not).

All contract upgrades are subject to a timelock (delay mechanism) managed by the `DelayProxyAdmin` contract. Once an upgrade is submitted via `submitImplementation`, it cannot be executed immediately — there is a minimum delay of 24 hours and a maximum delay of 30 days before it becomes effective. This mechanism ensures that users have sufficient time to review the proposed upgrade and redeem their funds if desired. Additionally, any change to the delay period itself is also governed by the same timelock mechanism.

In summary, both contract upgrades and strategy modifications are governed by strong dual-approval controls, ensuring that key operational decisions require coordinated authorization between the asset manager and distributor.

Conclusion

After Hashlock's analysis, the RockSolid project seems to have a sound and well-tested code base, now that our findings have been resolved or acknowledged to achieve full security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au